GPU Architecture & Computing — Semester Project

GPU-Accelerated Detection and Classification of Jigsaw Puzzle Pieces in Digital Images

Group 2

Ananthalakshmi Vinod Iyer
Jan-Hill Arganda
Jakob Olsacher
Tobias Ponesch
Philipp Sepin

June 23, 2026

Contents

1 Introduction

The goal of this project is to detect and classify jigsaw puzzle pieces in digital images. The input is an image containing several non-overlapping puzzle pieces on a background. Each piece is assumed to have four relevant edges, where each edge can be classified as a straight edge, a knob, or a hole. In this project, these edge types are denoted as `L`, `V`, and `C`. Since the pieces can appear in different rotations in the input image, the final class is normalized over rotations.

The pipeline has to solve several steps before such a class can be assigned. It first separates the puzzle pieces from the background, then detects connected regions, extracts the boundary of each piece, follows the boundary as an ordered contour, converts this contour into a radial signal, detects the four corners, classifies the four edges between them, and finally visualizes the result. The output is an annotated image that shows the detected contour, bounding box, piece number, and class label for every detected piece.

The project was implemented in two versions. The serial C++ version serves as the correctness and timing baseline and follows the structure of the provided Python reference implementation closely. The CUDA version keeps the same logical pipeline and output format, but changes the data flow where this is useful for GPU execution. Image-wide operations such as preprocessing and connected-component labeling are parallelized directly on the GPU, while several later stages are batched across all detected pieces to reduce per-piece overhead. Other stages, such as Moore contour tracing and final edge classification, remain on the CPU because their work is small, sequential, or branch-heavy.

The report therefore focuses on the complete pipeline rather than only on individual kernels. A GPU kernel can be fast in isolation, but the full application also pays for CUDA setup, memory allocation, host-device transfers, kernel launches, synchronization, and CPU-side work. For this reason, the implementation and the benchmarks consider not only total runtime, but also the cost of individual stages and the data movement around them. This makes it possible to identify which parts of the puzzle-classification pipeline benefit from CUDA and where the overhead dominates.

The following sections describe the stages of the pipeline, starting with image loading and preprocessing, followed by connected-component labeling, boundary and contour extraction, radial-signal generation, signal analysis, classification, and visualization. Afterwards, the report discusses the serial and CUDA pipeline integration, presents the benchmark results, and summarizes remaining limitations and possible future work.

2 Stage 1: Image Loading

The input image is loaded from disk into an RGB pixel buffer using `stb_image`, which decodes the source file into a flat row-major array of interleaved R, G, B bytes. This gives every later step the raw pixel data, and the image width and height to work with. At this stage, no processing happens.

Image loading is intentionally kept outside the measured pipeline time. Both the serial and CUDA implementations have to read the same input image, and decode time can fluctuate depending on storage, file format, and system load. Keeping it separate makes the comparison focus on the segmentation, classification, and visualization work. In a real end-to-end application, the CPU image decode phase could also be used to hide part of the CUDA startup latency, for example by starting CUDA context initialization with a small runtime call such as `cudaFree(nullptr)` while the image is being read. This optimization was not implemented because image loading is not part of the reported timing comparison.

3 Stage 2: Preprocessing and Cleaning

The preprocessing stage converts the RGB input image into a cleaned binary mask for the subsequent connected-component labeling stage. Both implementations perform the same logical sequence: grayscale conversion, normalization, Gaussian filtering, histogram construction, Otsu thresholding, binarization, and morphological opening.

3.1 Serial Implementation

Grayscale Conversion: The input image is loaded as an RGB image with three channels per pixel. The serial implementation iterates over all pixels and converts each RGB triplet to a floating-point

brightness value using the formula

$$Y = 0.299R + 0.587G + 0.114B.$$

The result is stored in a one-dimensional `std::vector<float>` with one entry per pixel. Each pixel is processed independently of its neighbours.

Normalization: The grayscale values are normalized to the range $[0, 255]$. The global minimum and maximum are first determined using `std::min_element` and `std::max_element`. Each grayscale value v is then transformed as

$$v' = \frac{v - v_{\min}}{v_{\max} - v_{\min}} \cdot 255.$$

If the intensity range is smaller than 10^{-5}, it is replaced by 1 to avoid division by zero. Normalization increases the available contrast and ensures that the histogram covers the complete 8-bit intensity range.

Gaussian Blur: A normalized two-dimensional Gaussian kernel is constructed using

$$G(x, y) = \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right).$$

The coefficients are divided by their total sum so that the kernel weights sum to one. The serial implementation then performs a direct two-dimensional convolution. For every output pixel, it traverses the complete $k \times k$ neighbourhood and calculates

$$I_{\text{blur}}(x, y) = \sum_{i=-r}^{r} \sum_{j=-r}^{r} G(i, j)\, I_{\text{norm}}(x + j, y + i),$$

where $r = k/2$. Coordinates outside the image are clamped to the nearest valid border coordinate.

The current pipeline configuration uses a 5×5 Gaussian kernel with

$$\sigma = 1.4.$$

This configuration was selected after segmentation experiments. The stronger blur reduces small gaps and weak dark markings near puzzle-piece boundaries more reliably than the previous configuration.

Histogram and Otsu Threshold: The blurred floating-point image is converted into a 256-bin histogram. Each brightness value is cast to an integer bin index, with the upper boundary clamped to ensure that values equal to 255 remain inside the histogram.

Otsu's method evaluates all possible thresholds and selects the value t_{Otsu} that maximizes the between-class variance

$$\sigma_b^2(t) = w_{\text{bg}}(t)\, w_{\text{fg}}(t)\, \left(\mu_{\text{bg}}(t) - \mu_{\text{fg}}(t)\right)^2.$$

Here, w_{bg} and w_{fg} are the background and foreground class weights, while μ_{bg} and μ_{fg} are their mean brightness values. The class weights and background sum are updated incrementally during the threshold scan.

The pipeline applies a fixed negative offset to the Otsu result:

$$t_{\text{used}} = t_{\text{Otsu}} - 15.$$

This offset expands weak bright foreground regions and reduces missing pixels along puzzle-piece boundaries. Thus, the threshold is image-dependent but also contains a fixed correction determined during segmentation experiments.

Binarization: The blurred image is converted into a binary image using the adjusted threshold:

$$B(x,y) = \begin{cases} 255, & I_{\text{blur}}(x,y) > t_{\text{used}}, \\ 0, & \text{otherwise.} \end{cases}$$

This polarity is used because the puzzle pieces are brighter than the background in the input images. The result is stored as an `ImageU8` object.

Morphological Cleaning: The binary image is cleaned using morphological opening. The serial implementation performs one erosion followed by one dilation with a dense rectangular 5×5 structuring element.

During erosion, an output pixel remains foreground only if every covered input pixel is foreground:

$$E(x,y) = \min_{(i,j)\in K} B(x+i, y+j).$$

Pixels outside the image are treated as background. The erosion removes small foreground artifacts, thin structures, and thresholding noise.

The following dilation restores a pixel to foreground if at least one covered pixel is foreground:

$$D(x,y) = \max_{(i,j)\in K} E(x+i, y+j).$$

Consequently, sufficiently large puzzle-piece regions are restored toward their original size, while small artifacts removed by erosion remain absent.

Morphological opening was retained after experiments with morphological closing. Although closing removed some small gaps, it introduced more severe topological errors. Nearby structures could merge, contours could form shortcuts between regions, and some pieces were no longer detected reliably. The segmentation was therefore tuned through the Gaussian parameters, the Otsu offset, and the minimum component area rather than by replacing opening with closing.

3.2 Parallel Implementation

The CUDA implementation preserves the logical operations of the serial pipeline while parallelizing the pixel-intensive work. The output representation is adapted for GPU processing: instead of returning a host-side `ImageU8` object, the CUDA implementation writes the cleaned binary mask directly to a device-side `uint8_t` buffer. In both implementations, background and foreground pixels are represented by 0 and 255, respectively.

The main preprocessing optimizations are grid-stride processing, parallel minimum and maximum reduction, constant-memory storage for Gaussian coefficients, block-local histogram construction, fusion of binarization and erosion, shared-memory tiling, and device-resident transitions between pipeline stages.

CUDA buffer allocation and lifetime optimizations are discussed separately in the pipeline integration section.

Parallel Grayscale Conversion: RGB-to-grayscale conversion is implemented as a one-dimensional grid-stride CUDA kernel. Each thread processes one or more pixels and computes the same weighted sum as the serial implementation.

Since each output pixel is independent, no synchronization between threads is required. Consecutive threads process consecutive RGB pixels and write consecutive grayscale values, producing a regular global-memory access pattern. The grid-stride loop also allows one launch configuration to process images of different sizes.

Parallel Normalization: The global minimum and maximum grayscale values are computed on the GPU using `thrust::minmax_element`. This replaces the two sequential scans of the serial implementation with a parallel reduction.

A single-thread CUDA kernel stores the resulting values in a two-element device buffer. A second grid-stride kernel then normalizes all grayscale values in parallel:

$$I_{\text{norm}}(p) = \frac{I(p) - I_{\min}}{I_{\max} - I_{\min}} \cdot 255.$$

The same protection against an intensity range close to zero is used as in the serial implementation. The extrema remain on the GPU, avoiding a host round trip between the reduction and normalization operations.

Constant-Memory Gaussian Kernel: The 5×5 Gaussian coefficients are computed on the host once per preprocessing run using the same formula and $\sigma = 1.4$ as in the serial implementation. The normalized coefficient table is then uploaded to CUDA constant memory.

Constant memory is suitable for this operation because the table is small, read-only, and accessed identically by all threads. When threads within a warp request the same coefficient, the constant cache can broadcast it instead of performing an individual global-memory load for every thread.

Parallel Gaussian Blur: The Gaussian blur is implemented as a one-dimensional grid-stride kernel. Each thread calculates one or more output pixels by traversing the complete 5×5 neighbourhood. Border coordinates are clamped in the same manner as in the serial implementation.

Image samples are read from global memory, while the Gaussian coefficients are read from constant memory. The operation remains a direct two-dimensional convolution rather than two separable one-dimensional passes. Its primary acceleration therefore comes from processing independent output pixels concurrently and from caching the shared coefficient table in constant memory.

Block-Local Histogram Construction: A naive parallel histogram would cause all threads to update the same 256 bins in global memory, resulting in considerable atomic contention. The implementation therefore creates one local 256-bin histogram in shared memory for every CUDA block.

Each block performs the following operations:

1. The threads cooperatively initialize the shared histogram.
2. The image pixels assigned to the block are processed using a grid-stride loop.
3. Pixel values are accumulated with shared-memory atomic operations.
4. After synchronization, the local bins are merged into the global histogram using global atomic operations.

Most pixel updates therefore affect the faster block-local histogram. Only 256 aggregated values per block must be merged into global memory.

The histogram grid is limited to at most 1024 blocks. Beyond this point, more blocks would mainly increase the number of global merge operations without providing a proportional increase in useful parallel work.

Device-Side Otsu Threshold: The Otsu threshold scan is executed by one CUDA thread. Although this operation is sequential, it processes only 256 bins. A more complex parallel implementation would introduce additional synchronization and reduction overhead for a very small amount of work.

Keeping the threshold calculation on the GPU avoids copying the histogram to the CPU and transferring the selected threshold back to the device. The Otsu result remains device-resident and is consumed directly by the cleaning stage.

The negative offset is applied inside the fused binarization and erosion kernel:

$$t_{\text{used}} = t_{\text{Otsu}} - 15.$$

Fused Binarization and Erosion: Binarization is not launched as an independent CUDA kernel. Instead, the threshold comparison is fused directly into the erosion kernel.

While loading an image tile, the threads evaluate

$$B(x,y) = \begin{cases} 255, & I_{\text{blur}}(x,y) > t_{\text{used}}, \\ 0, & \text{otherwise.} \end{cases}$$

The resulting binary values are placed directly in shared memory. The erosion operation then consumes the thresholded tile without first storing a complete binary image in global memory.

This fusion removes one kernel launch as well as one full-image global-memory write and the corresponding read by a separate erosion kernel.

Shared-Memory Tiling: The erosion and dilation kernels use 16×16 thread blocks. Since the 5×5 structuring element has a radius of two pixels, every block cooperatively loads its central image tile together with a two-pixel halo into shared memory.

For a 16×16 output region, the shared tile contains

$$(16 + 2 \cdot 2) \times (16 + 2 \cdot 2) = 20 \times 20$$

binary values.

After loading the complete tile, the threads synchronize before evaluating their neighbourhoods. Adjacent output pixels use strongly overlapping 5×5 neighbourhoods. Shared-memory tiling therefore allows neighbouring threads to reuse input values instead of repeatedly loading the same pixels from global memory.

Out-of-image values are inserted into the tile as background, matching the serial morphology implementation.

Tiled Erosion: Each thread examines its 5×5 neighbourhood in shared memory. The output pixel remains foreground only if all neighbourhood values are foreground.

The scan terminates as soon as a background value is encountered. This early exit avoids unnecessary comparisons for neighbourhoods that clearly do not satisfy the erosion condition.

Tiled Dilation: The dilation kernel uses the same 16×16 block geometry and two-pixel halo. Threads cooperatively load the eroded binary mask into shared memory before examining their 5×5 neighbourhoods.

An output pixel is set to foreground as soon as one foreground neighbour is found. This operation also terminates its neighbourhood scan early. Reusing the same tile geometry and border handling for both operations keeps the CUDA result consistent with the serial 5×5 morphological opening.

4 Stage 3: Connected Component Labelling

4.1 Serial Implementation

Floor-Fill Labelling The cleaned mask is scanned pixel by pixel. Whenever an unlabeled foreground pixel is found, a queue-based flood fill (breadth-first search) spreads out to every foreground pixel reachable through its 8 neighbours, assigning all of them the same new integer label. While the flood fill is running, it keeps an incrementing pixel count (for the region's area) and a running minimum & maximum x and y coordinates seen so far (for the region's bounding box), so both are already known the moment the area is finished without iterating over it again.

Minimum-Area Filtering Once a blob's flood fill finishes, its area is compared against a minimum threshold. The current pipeline uses a base value of 45000 pixels for the reference image height of 5100 pixels and scales this threshold quadratically with the image height for other resolutions. Blobs above the threshold are kept as separate `Region` objects with their own label, area, and bounding box, while blobs below the threshold are assumed to be leftover noise from thresholding or cleaning and are erased by resetting their labels back to 0.

4.2 Parallel Implementation

Profiling the pipeline showed that connected-component labelling was dominated by repeated label propagation. An earlier GPU version that was implemented mirrored the serial flood fill by giving every foreground pixel its own initial label and propagating the minimum label to each pixel's neighbours one step per kernel launch; a label could then need as many full-image launches as the diameter of its component to fully spread, which is costly for large puzzle pieces. The implementation was therefore replaced with a Block-Based Union-Find (BUF) labelling algorithm following Allegretti et al. [1], designed specifically for 8-connected GPU labelling: since every foreground pixel inside a 2×2 block is automatically connected to its block-mates under 8-connectivity, BUF operates on blocks instead of individual pixels, cutting the number of labels, memory accesses, and union-find merges by close to a factor of four.

Concretely, the kernel pipeline makes four passes over a `parents` array sized to the full image. `init_block_labels_kernel` assigns one thread per 2×2 block; if any of its four pixels is foreground, the block is seeded with its own top-left pixel index as a self-referential root. `Merge_block_labels_kernel` assigns one thread per block again and checks its right, bottom, and two diagonal neighbouring blocks for pixels that actually touch under 8-connectivity (not merely adjacent blocks). Whenever two blocks touch, their tree roots are merged with a lock-free union operation that walks each side's parent chain to its root and uses an atomic compare-and-swap to attach the larger root under the smaller one. `Compress_block_labels_kernel` then gives one thread to every entry in `parents` and flattens it directly to its final root (path compression), so the next kernel can read any label in constant time instead of walking a chain. Finally, `final_labels_from_blocks_kernel` assigns one thread per image pixel, looks up which 2×2 block it belongs to, and writes that block's already-compressed root as the pixel's label, expanding the compact block-level labelling back out to the per-pixel label image the rest of the pipeline expects.

Region statistics are then built with three further kernels in place of the serial flood fill's single running count. `init_region_stats_kernel` zeroes one area/bounding-box slot per 2×2 block, `accumulate_region_stats_kernel` assigns one thread per pixel that atomically increments its block-root's area and tightens its bounding box with `atomicMin`/`atomicMax`, and `emit_regions_kernel` discards slots whose area falls below the same scaled minimum-area threshold used by the serial implementation and atomically appends the rest into a compacted `Region` array. The surviving regions are copied back to the host and sorted by (y, x) so piece ordering matches what a raster-order flood fill would produce, keeping downstream piece numbering identical between the two pipelines.

5 Stage 4: Boundary Extraction

5.1 Serial Implementation

Within that cropped window, every pixel belonging to the region's label is checked against its 8 neighbours. If any neighbour is outside the image or carries a different label (background, or a different piece), the pixel is marked as a boundary pixel; otherwise it is an interior pixel and is left blank. The output is a thin, one-pixel-wide outline of just that one piece.

5.2 Parallel Implementation

All regions are processed in one batched launch instead of one launch per region. A host-side planning step first computes, for every detected region, a one-pixel-padded bounding box clamped to the image edges and records its offset into a single flat output buffer sized to the sum of every padded box's area; the single largest box area is also recorded to size the kernel's launch grid. The batched kernel then maps `blockIdx.y` to a region and lets `blockIdx.x`/`threadIdx.x` range over local positions inside that region's padded box, so smaller regions simply have threads exit immediately once their local index exceeds that

region's own area while still sharing the same rectangular launch as the largest region. Each surviving thread maps its local index back to absolute image coordinates and applies the identical 8-neighbour boundary test used by the serial implementation, a foreground pixel with at least one neighbour outside the image or carrying a different label, writing 255 into the flat output buffer at that region's offset. The whole buffer is zero-initialized once for the entire batch with a single `cudaMemset` before the kernel runs, since not every position inside a region's padded box belongs to that region.

6 Stage 5: Contour Extraction

6.1 Serial Implementation

Moore neighbour Tracing Starting from the first nonzero pixel found by scanning top-left to bottom-right, the algorithm walks around the contour one step at a time, checking its unvisited neighbours in a clockwise order. This matches the behaviour of the Python baseline. It terminates when it arrives back at the starting pixel or no unvisited neighbours remain. This produces an ordered list of contour points. Since each next point depends on the current pixel and the arrival direction, the algorithm is inherently sequential.

Chain Simplification The ordered contour is then reduced by dropping contour points that lie along straight lines between two other contour points, keeping only points where the contour changes direction. This is also inherently sequential because each point's relevance depends on its two immediate neighbours in traced order. After simplification, coordinates are swapped from (row, col) to (x, y) to match the convention used in the Python baseline.

6.2 Parallel Implementation

There are parallel alternatives to Moore neighbourhood tracing, such as splitting the image into chunks or processing each pixel independently, but all of these approaches return an unordered set of contour points. Reconstructing the correct order from such a set is highly complex, so we run the exact same sequential algorithm as the serial implementation, once per detected piece, on the host. But since different puzzle pieces are fully independent of each other, we use OpenMP to run one CPU trace per piece concurrently.

After tracing, `build_batched_contours` packs all piece contours into a flat structure-of-arrays layout that is passed to the following CUDA stages:

- `points`: all contour points of all pieces concatenated back-to-back
- `offsets[i]`: start index of piece i in the flat `points` buffer
- `lengths[i]`: number of contour points of piece i
- `max_length`: length of the longest contour, used for kernel grid sizing

This layout allows every subsequent CUDA stage to process all pieces in a single batched kernel launch.

7 Stage 6: Contour Smoothing

7.1 Serial Implementation

A simple moving-average filter with a default window size of 5 is applied to the contour points. Each point's coordinates are replaced with the average of the window of nearest neighbours centered on that point. Since the contour is a closed loop, windows that extend beyond the edge wrap around to the other side. This removes small artefacts before the next stages.

7.2 Parallel Implementation

Once all piece contours are packed into the flat batch buffer, a single kernel call smooths all of them at once. Pieces are selected by `blockIdx.y`. Threads whose local index exceeds either `max_length` or the piece's actual length n exit immediately, so pieces of different lengths can share one launch. Each active thread computes one smoothed output point by summing the same window of neighbours as the serial filter, using modulo n to handle the closed contour, matching the serial guard clause exactly.

The contours are uploaded to the device once at the start of this stage. The smoothed result remains on the device after the kernel and is passed directly to the next stages without an intermediate device-to-host transfer.

8 Stage 7: Enclosing Circle Approximation

8.1 Serial Implementation

A loop over the smoothed contour coordinates finds the minimum and maximum x and y values of all contour points. The center of the enclosing circle is then approximated as the midpoint of this bounding box.

$$x_c = \frac{x_{\max} + x_{\min}}{2}, \qquad y_c = \frac{y_{\max} + y_{\min}}{2}$$

Note that this is only an approximation, as this is not the same as the center of the true minimum enclosing circle. A second loop then computes the maximum Euclidean distance from this center to any contour point, which is returned as the radius. This is done in analogy to the Python Baseline.

8.2 Parallel Implementation

The parallel implementation computes only the center, not the radius, since no later stage actually uses it, even though it was included in the Python baseline.

An earlier version solved this with reductions using Thrust. However, reducing all four values (min x, max x, min y, max y) with Thrust requires four separate `reduce_by_key` calls and key arrays. A better solution is a custom kernel that reduces all four values simultaneously in a single pass and processes all pieces in one launch. The kernel assigns one block per piece with 256 threads each. Each thread performs a strided scan over its piece's contour points to accumulate local min and max values per thread. These are written to shared memory, after which a parallel tree reduction finds the block-wide minimum and maximum in $\log_2(256) = 8$ steps. Thread 0 then computes the center of each piece and writes it to the flat output array. The shared memory arrays are sized to exactly 256 entries, matching the fixed block size.

Warp shuffles (`__shfl_down_sync`) would make the reduction faster by avoiding shared memory and `__syncthreads` barriers, but this stage takes under 10 μs and is not a bottleneck.

9 Stage 8: Radial Signal Generation

9.1 Serial Implementation

The radial signal s_i is acquired by computing the Euclidean distance from each smoothed contour point to the circle center computed in the stage before. This is done by looping over all contour points.

$$s_i = \sqrt{(x_i - x_c)^2 + (y_i - y_c)^2}$$

9.2 Parallel Implementation

A batched kernel computes the radial signal for all pieces in one launch, using the same grid layout as Stage 6. Pieces are selected by `blockIdx.y`. Threads beyond `max_length` or beyond the piece's actual length n exit immediately. Each remaining thread independently reads its contour point from the flat buffer, loads the precomputed center for that piece, and writes the Euclidean distance $\sqrt{dx^2 + dy^2}$ to the flat signal buffer at the same index. The kernel is embarrassingly parallel, as every thread is fully independent.

After the kernel, the flat signal buffer is copied from device to host. This is done because Stage 11 (edge classification) runs on the CPU and requires the raw radial signal on the host.

10 Stage 9: Signal Smoothing

10.1 Serial Implementation

We then use a centered averaging window for smoothing the radial signal obtained from the previous step using the following formula using odd numbers of window lengths k for our smoothed signal $\tilde{s}_i$:

$$\tilde{s}_i = \frac{1}{k} \sum_{j=-\frac{k-1}{2}}^{\frac{k-1}{2}} s_{i+j}$$

10.2 Parallel Implementation

For the signal smoothing we use a batched kernel, following the same grid configuration as the Stage 8 radial kernel. Each thread maps to a single output sample $\tilde{s}_i$, computing the centered average independently across a 2D grid. Threads corresponding to indices beyond a piece's length n exit immediately, allowing pieces of varying lengths to share a rectangular launch.

The kernel implements circular boundary conditions by wrapping indices modulo n within each thread. Because each output $\tilde{s}_i$ is a function of a static input neighborhood and requires no communication with other threads, the computation is entirely data-parallel and requires no synchronization primitives.

To minimize overhead, the smoothed signal remains in device memory for immediate use as input to the Stage 10 local-maxima kernel. A single host copy is made once, replacing the per-piece copy that a non-batched design would require. The main parts responsible for speedup was achieved through batching the single puzzle piece signals a non-batched design would issue one kernel launch and one device-to-host copy per piece.

11 Stage 10: Peak Detection

11.1 Serial Implementation

Peak detection operates on the smoothed signal $\tilde{s}$ in three filtering stages followed by a greedy selection. First, all local maxima are collected as candidates: index i qualifies when $\tilde{s}_i \geq \tilde{s}_{i-1}$ and $\tilde{s}_i > \tilde{s}_{i+1}$ with indices taken modulo n. Each candidate is then tested against three criteria.

1. Its *prominence*
$$p_i = \tilde{s}_i - \max\left(\min_{j \in L_i} \tilde{s}_j,\ \min_{j \in R_i} \tilde{s}_j\right),$$
where L_i (R_i) is the set of indices reached by walking left (right) from i before encountering a higher value, must satisfy $p_i \geq p_{\min}$.

2. Its *sharpness*
$$\sigma_i = \tilde{s}_i - \tfrac{1}{2}(\tilde{s}_{i-w} + \tilde{s}_{i+w}), \quad w = 20,$$
must satisfy $\sigma_i \geq \sigma_{\min}$.

3. Finally, the circular *distance* $d(i,j) = \min(|i-j|,\ n - |i-j|)$ the most recently kept peak must be at least $d_{\min}$. If two candidates fall within that distance, the taller one is kept.

From the surviving candidates exactly four corners are chosen greedily: the highest-valued peak is selected first, and each subsequent corner is the candidate that maximises its minimum circular distance to all already chosen corners.

11.2 Parallel Implementation

For the peak detection the work partition is split across the GPU and the CPU, where only the local-maxima candidate search is moved to the GPU. The prominence, sharpness, distance, and greedy four-corner selection stages remain unchanged on the CPU, operating on the same flat `Signal` buffer used by the serial implementation.

The kernel evaluates the local-maxima condition for all pieces simultaneously. Using the same 2D grid configuration as previous stages, each thread maps to a unique contour position. Threads exceeding a piece's length n exit safely, allowing for efficient processing of varying-length pieces within a single rectangular launch.

Each thread reads $\tilde{s}_i$ and its circular neighbours $\tilde{s}_{i-1}$ and $\tilde{s}_{i+1}$. A binary mask is written: 1 if $\tilde{s}_i \geq \tilde{s}_{i-1}$ and $\tilde{s}_i > \tilde{s}_{i+1}$, and 0 otherwise. This replicates the serial implementation's asymmetric comparison, ensuring consistent resolution of flat-topped peaks.

The resulting mask array is copied to host memory in a single transfer. The CPU then performs a single pass per piece to convert these masks into a list of candidate indices. From this point on, the algorithm is unchanged from the serial implementation.

12 Stage 11: Edge Classification

12.1 Serial Implementation

Each of the four edges runs along the raw signal between two consecutive corners c_e and c_{e+1}. The signal values at the two cut-off points, $v_{\text{start}} = s_{c_e+m}$ and $v_{\text{end}} = s_{c_{e+1}-m}$, serve as a local baseline representing the expected signal level of a flat edge near the corners. $m = \frac{len(edge)}{5}$

The interior region is then scanned for its highest value $s_{\max}$ and lowest value $s_{\min}$. A tolerance $\tau = \alpha \cdot \max(v_{\text{start}}, v_{\text{end}})$ scales with the local signal level so that large and small pieces are treated consistently. The edge is classified as:

$$\ell_e = \begin{cases} \texttt{V} & \text{if } s_{\max} > \max(v_{\text{start}}, v_{\text{end}}) + \tau \quad \text{(knob: signal rises above baseline)} \\ \texttt{C} & \text{else if } s_{\min} < \min(v_{\text{start}}, v_{\text{end}}) - \tau \quad \text{(hole: signal dips below baseline)} \\ \texttt{L} & \text{otherwise} \quad \text{(straight: signal stays near baseline).} \end{cases}$$

The four labels are concatenated into a four-character signature (e.g. `LVLC`). A pre-built lookup table covers all $3^4 = 81$ possible signatures. The category is determined by the number of straight edges: `C` (2), `E` (1), `I` (0), corresponding to corner, edge and interior piece types.

12.2 Parallel Implementation

For this we decided to retain the edge classification on the CPU in order to avoid the overhead of additional GPU kernel launches and data transfers. This part in the parallel pipeline follows the serial one.

13 Stage 12: Visualization

13.1 Serial Implementation

After all pieces have been detected and classified, the visualization stage turns the numerical result back into an image that can be inspected manually. In the serial implementation, this is done on a copy of the loaded RGB buffer, so the original input image remains unchanged. The renderer then walks over the detected pieces and draws the relevant information directly into this copied image: the contour is shown in red, the bounding box is shown in light green, and the class label is written above the box.

The contour is rendered by connecting neighbouring contour points with **Bresenham's line algorithm**, which maps a straight line segment onto integer pixel coordinates. Since the contour represents a closed puzzle-piece outline, the last contour point is connected back to the first one. The bounding box is drawn with the same line drawing routine, but instead of using a thin one-pixel rectangle, its thickness is scaled with the image height so that it remains visible on the large benchmark images. The rectangle is expanded outward from the detected region, which keeps the inside of the puzzle piece visible and makes it easier to compare the green box with the red contour and the actual object boundary.

The label contains the display number and the detected class, for example `P. 1 ; Class C`. The displayed piece number is based on the final piece order rather than the internal component label, so the overlay uses compact labels such as `P. 1`, `P. 2`, and so on.

The label text is rendered with `stb_truetype`; each glyph is rasterized into a greyscale coverage bitmap and blended into the image using the glyph coverage value:

$$p'_c = \alpha f_c + (1 - \alpha) p_c,$$

where f_c is the text colour, p_c is the previous image colour, and α is the glyph opacity. The font size and padding are scaled with the image resolution, which keeps the labels readable after resizing.

The finished overlay is written as BMP in both the serial and CUDA pipelines. Although the lower-level writer can also write PNG, the current pipeline always constructs a `.bmp` output path. This was done because PNG compression added a large amount of extra runtime during profiling, while BMP writing was much faster. In addition, storing the final image is file I/O and can fluctuate for reasons that are not directly related to the segmentation and classification pipeline, similar to image loading. Therefore the final image is written after the measured pipeline timer has stopped, and the visualization timing only contains the drawing work in memory.

13.2 Parallel Implementation

The CUDA pipeline creates the same kind of overlay, but the drawing itself still runs on the CPU. This was a deliberate choice, because the visualization work is very different from the earlier image-wide CUDA stages. Drawing a few contours, rectangles, and text labels is small, branch-heavy work, while the text rendering is already handled by `stb_truetype` on the host. In addition, the final image is saved with `stb_image_write`, which expects host memory, so a CUDA renderer would still need to copy the completed image back to the CPU before writing it. For this stage, that extra setup and transfer cost would not be justified.

The parallel implementation therefore keeps the renderer on the CPU, but it avoids an unnecessary full-image copy. Since the CUDA pipeline writes the overlay immediately after drawing it, the renderer can draw directly into the already loaded host RGB buffer. In other words, the serial version creates a separate overlay image, whereas the CUDA version modifies the host image in-place before saving it.

When OpenMP is available, the CUDA visualization path also distributes the drawing over the detected pieces, so different pieces can be rendered by different CPU threads. Each thread draws one piece's red contour, green bounding box, and white label. The visual conventions remain the same as in the serial version, including the display numbering, the line thickness scaling, the outward boxes, and the default BMP output. Optional NVTX ranges are only used for profiling, for example to separate overlay drawing from image writing.

14 Pipeline

14.1 Serial Implementation

The serial pipeline is the reference implementation that the CUDA version is compared against. Its structure is strongly inspired by the provided Python reference implementation, but the individual stages were rewritten in C++ so they could be timed consistently and compared to the CUDA version. It can be run on either a single image or a folder of images. When a folder is given, the supported image files are collected and processed in sorted order, which makes benchmark runs reproducible. For every image, the input is first loaded as an RGB image with `stb_image`. The pipeline timer starts only after this image loading step, and the final overlay file is written only after the timer has stopped. This keeps image decoding and file output outside the measured processing time.

After loading the image, the serial implementation follows the stage order described in the previous sections. First, preprocessing converts the RGB image into a cleaned binary mask. Connected-component labeling then creates the label image and extracts the detected regions. The minimum region area is scaled with the square of the image height, so the same parameter remains meaningful for different benchmark resolutions.

The rest of the serial pipeline processes the detected regions one after another. For each region, a cropped boundary mask is extracted from the label image and converted into an ordered contour. Because the boundary mask is local to the cropped region, the contour points are shifted back into full-image coordinates before they are used by later stages. The contour is then smoothed, an approximate center

is computed, and the contour is converted into a radial signal. This signal is smoothed again for peak detection, the four corner peaks are selected, and the raw radial signal together with those corners is used to classify the four edges. Finally, the edge signature is mapped to the piece class through the lookup table and the completed `PuzzlePiece` is stored in the result vector.

This structure is intentionally simple. Each detected piece is processed completely before the next one starts, and intermediate data is kept in normal CPU containers such as `std::vector` and the project image types. That makes the serial path easy to follow and useful as a correctness baseline, while also making it clear where the CUDA version later changes the data flow by batching work across pieces. After all pieces have been processed, the serial visualization stage draws the overlay into a separate output image, the total pipeline timer is stopped, and the overlay is written as a BMP file.

14.2 Parallel Implementation

14.2.1 Overall Structure and CUDA Setup

The CUDA pipeline follows the same logical stages as the serial implementation, but it is not a direct line-by-line translation. The important change is that the pipeline owns the main device buffers and passes them through the stages, instead of letting every stage allocate memory, copy inputs, copy outputs, and return large temporary results on its own. This was one of the main integration steps of the project, because the first individual implementations were written with different assumptions about data layout and memory ownership. Some stages used flat arrays, some used nested containers or custom image types, and several functions managed their own temporary buffers internally. When these parts were first connected in a serial-style loop over all pieces, the CUDA version spent a large amount of time on repeated allocations, copies, kernel launches, and synchronizations.

In the current pipeline, image loading is still done on the CPU with `stb_image`, and the measured timer starts only after loading has finished, just as in the serial version. The first CUDA stage is an explicit `CUDA setup` range, which includes the first CUDA runtime work, image-sized device allocations, preprocessing and component-labeling scratch allocations, and the upload of the RGB image to the device. The large device buffers are owned by `thrust::device_vector`, while kernels receive raw pointers obtained from those vectors after allocation. This keeps ownership clear in the pipeline, but still allows normal CUDA kernels to use plain device pointers.

Not all allocation sizes are known at the beginning of the image. For example, boundary extraction depends on the number and size of detected regions, and the batched contour and signal buffers depend on the lengths of the contours found by Moore tracing. These allocations are therefore performed later, when the required sizes are known, but their cost is still charged to the `CUDA setup` timing bucket. This makes the later stage timings describe mostly the work of the stage itself, instead of mixing computation with one-time buffer allocation. The same timing boundary as in the serial pipeline is kept at the end: drawing the overlay is measured, but writing the final BMP file happens after the total pipeline timer has stopped.

The main algorithm parameters are fixed near the pipeline code as `constexpr` values. This avoids passing rarely changed tuning values through runtime option structures and lets the compiler treat them as constants where possible. The serial and CUDA versions still have to keep matching values manually; a shared configuration header would have been a cleaner way to express that relationship. In the current implementation, runtime options are kept to values that actually change between runs, such as the input path and output directory.

The standalone CUDA executable can also receive a folder path. In that case it collects all supported image files, sorts them, and calls `run_cuda` once per image. This means folder mode benefits from staying in the same process with an already initialized CUDA runtime, but it is not yet a true persistent-buffer pipeline: the main `thrust::device_vector` buffers and scratch structures are still owned by one `run_cuda` call and are created again for the next image.

14.2.2 GPU Stages and Host Stages

The first stages are image-wide operations and therefore fit the GPU well. Preprocessing reads the uploaded RGB image and writes the cleaned binary mask directly in device memory. Connected-component labeling then runs on the device using the block-based union-find implementation, and the following region-build step filters small components with the same height-scaled minimum-area threshold as the

serial pipeline and copies only compact `Region` metadata back to the host. Boundary extraction is also performed as one batched CUDA stage. A host-side plan first computes the cropped bounding boxes and offsets for all regions, then the CUDA kernel extracts all piece boundary masks into one flat output buffer.

Some stages intentionally remain on the CPU. The most important example is Moore contour tracing. The trace must follow the boundary in order, and each next point depends on the current point and search direction. A direct GPU kernel would not solve this well without changing the algorithm. Therefore the CUDA pipeline copies the packed boundary masks back to the host and runs the same contour tracing logic there. The work is still parallelized over pieces with **OpenMP** when available, because each piece owns its own boundary mask and output contour.

Peak detection and edge classification are also split carefully. The local maximum mask for peak detection is computed with a batched CUDA kernel, but the prominence, sharpness, distance filtering, and greedy selection of the four corners stay on the CPU. Edge classification is also kept on the CPU in the current pipeline. Because these CPU stages still need signal data, the radial signal, smoothed signal, and local-maximum mask are each copied back once as flat batched arrays instead of being copied once per piece. An isolated GPU edge-classification experiment reduced one intermediate radial-signal copy, but the additional launch and tiny transfer overhead made the total result worse. This is why the pipeline does not move small pieces of branch-heavy work to CUDA just for the sake of using the GPU. If the remaining stages were redesigned so that the whole contour and signal analysis path stayed on the device, moving more of this work to CUDA could become beneficial, because the intermediate host copies would disappear. We did not implement this larger redesign, because it would require finding or developing parallel algorithms for the currently sequential parts of the pipeline. After edge classification, the corner indices, edge labels, and class label from the lookup table are attached back to the corresponding `PuzzlePiece` objects.

This split between GPU and CPU stages is also visible in the transfer-overhead profile. Image-wide stages such as preprocessing and connected-component labelling are dominated by GPU computation, while stages that cross the CPU/GPU boundary show a more visible transfer cost. Boundary extraction is the clearest example, because the packed boundary masks have to be copied back for CPU-side Moore tracing. The later signal stages are much smaller in absolute runtime, but their device-to-host copies are still noticeable compared to the short kernel times. This supports the design decision to batch these transfers, and it also shows why moving only one small branch-heavy step to CUDA did not automatically improve the full pipeline.

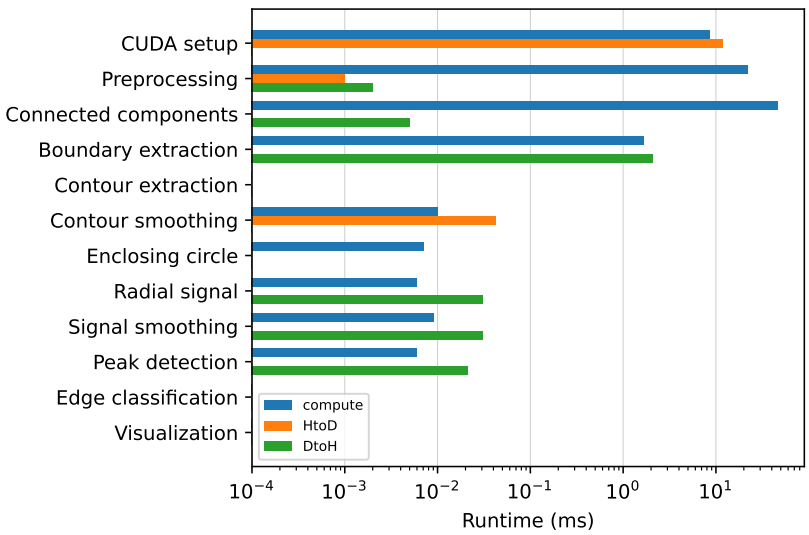


Figure 1: CUDA computation and host-device transfer overhead for the individual pipeline stages.

The plot also shows that future work should focus less on isolated small kernels and more on keeping complete stage groups device-resident, so that intermediate transfers disappear.

14.2.3 Batched Contour and Signal Processing

The largest pipeline-level optimization was replacing the initial per-piece CUDA style with batched buffers. The early combined version still looked very close to the serial pipeline: it looped over all pieces, called a CUDA function for one piece, and that function often performed its own allocations, copies, and synchronizations before returning a result. This was correct, but it caused a large amount of overhead, especially when many pieces were present.

The current implementation instead packs all contours into flat arrays. For each piece, the pipeline stores an offset and a length, and all contour points are stored consecutively in one buffer. The same idea is used for radial signals and intermediate signal-analysis data. With this layout, contour smoothing, enclosing-circle center approximation, radial-signal generation, signal smoothing, and local-maximum detection can each run as one batched stage over all pieces. Threads that map beyond the true length of a shorter piece simply exit, while longer pieces use more work from the same launch. This replaces many small per-piece launches and copies with a few larger launches that have enough work to justify the GPU overhead.

The stage interfaces were changed accordingly. Instead of returning large vectors from device helper functions, the functions write into buffers owned by the pipeline or by scratch structures. This made it possible to keep reusable device buffers alive across stages and to remove avoidable transfers. One example is the enclosing-circle center computation: the centers are written directly into device scratch memory used by the next stage, so the older unused host copy of the centers was removed. More generally, the pipeline now decides when data has to cross the host-device boundary, instead of every subfunction making that decision independently.

14.2.4 Memory Layout and Coalescence

The final CUDA pipeline uses a small number of top-level image buffers together with stage-specific scratch structures. Image-sized buffers such as the RGB input, cleaned mask, and compact label image are visible directly in `run_cuda`. Smaller groups of related temporary buffers are stored in scratch structs for preprocessing, connected components, boundary extraction, contour processing, and signal processing. This keeps allocation ownership in one place while still avoiding a single large wrapper object that would hide the main data flow.

Most image-wide CUDA stages use flat row-major arrays, which means that neighboring threads usually access neighboring pixels in memory. This is beneficial for memory coalescence in the grayscale conversion, normalization, Gaussian blur output, morphological cleaning, and most batched contour and signal kernels. The morphology kernels additionally use shared-memory tiles with a halo region, so repeated neighborhood reads are served from shared memory instead of being loaded from global memory again and again.

The less regular parts of the pipeline are mainly caused by the algorithms rather than by a simple memory-layout mistake. Connected-component labeling uses a block-based union-find approach, where parent chasing and merging do not follow a perfectly linear access pattern. The later region-statistics kernel reads the label image linearly, but the updates are written with atomics to component-dependent slots, which can lead to scattered writes and contention. Boundary extraction is also less regular, because each thread has to inspect neighboring labels and the amount of useful work depends on the shape of the component.

There are still possible improvements. The Gaussian blur currently performs overlapping stencil reads from global memory, so a shared-memory tiled version or a fused preprocessing kernel could improve memory reuse. The RGB input is stored as interleaved three-byte data, which is acceptable but not as clean as an aligned RGBA or `uchar4`-style layout. For contour and signal data, an internal struct-of-arrays layout could also be tested on the GPU side, while keeping the public `CoordinateVector` interface unchanged for the serial code, tests, visualization, Moore tracing, and GPU packing code. Overall, the current implementation has mostly coalesced access in the regular image-wide stages, while the remaining weaknesses are concentrated in irregular component-based processing and in places where more shared-memory reuse could still be added.

14.2.5 Timing and Instrumentation

Several optional features are controlled with compile-time flags. `SUB_TIMINGS` enables sub-stage timing, `PERSIST_TIMINGS` writes CSV rows for benchmark analysis, `DEBUG_LEVEL` controls console summaries,

and `ENABLE_NVTX` adds named ranges for Nsight profiling. When these features are disabled, the related printing, timing persistence, or NVTX instrumentation is compiled out or reduced to minimal overhead. Timing CSV rows include the image name, resolution, piece count, run number, total time, CUDA setup time, and the individual stage times, which made it possible to compare single-image and folder-mode behavior consistently.

The effect of the pipeline cleanup can be seen clearly in the Nsight Systems profiles from the development process. Comparing an early integrated CUDA pipeline profile with a later optimized profile, the number of CUDA operations dropped strongly. In the table, the runtime API row is the broad total of CUDA runtime calls, while the rows below it show the most relevant groups inside that total:

Recorded CUDA runtime activity	Old profile	New profile	Reduction
All runtime API calls	6330	166	97.4%
Kernel launches	1318	52	96.1%
Memory-copy calls	1084	13	98.8%
Synchronization calls	2062	39	98.1%
`cudaMalloc` calls	932	29	96.9%
`cudaFree` calls	932	29	96.9%

Table 1: CUDA runtime call counts before and after the pipeline-level batching and memory-ownership cleanup. The indented rows are included in the total runtime API call count; the total also contains small remaining calls such as `cudaMemset` and module queries.

These numbers show why the pipeline design mattered as much as the individual kernels. The optimization was not only that some kernels became faster, but also that the pipeline stopped launching, copying, allocating, and synchronizing for every small piece of work. This reduced the overhead around the kernels and made the CUDA implementation behave more like a single connected pipeline.

15 Benchmarking

The benchmark evaluation compares the end-to-end runtime of the serial C++ implementation with the CUDA implementation. In addition to the total runtime, the measurements contain a breakdown of the individual pipeline stages. The benchmark therefore shows both the overall scaling behaviour and which parts of the image-processing pipeline benefit most from GPU acceleration.

15.1 Benchmark data set and procedure

The benchmark data set consists of 13 manually counted puzzle images containing between 31 and 65 pieces. Each image is available at four resolutions. This separates the influence of image size from the influence of the number of detected pieces.

Test case	Resolution	Pixels	Puzzle pieces
1	1024×1409	1.44 million	31–65
2	2048×2817	5.77 million	31–65
3	4096×5635	23.08 million	31–65
4	5100×7016	35.78 million	31–65

Table 2: Benchmark resolutions and workload range. Each resolution contains the same 13 source images.

For every image and resolution, ten independent runs were recorded for both implementations. This results in 260 single-image measurements per implementation. The reported value for one image is the median of its ten runs; values aggregated over a resolution are the median over the 13 images. The median was selected because it is less sensitive to process start-up and CUDA context initialization outliers than the arithmetic mean.

The repository additionally contains a folder-mode benchmark at 5100×7016. In this mode, all 13 images are processed by one program invocation, again repeated ten times. Folder mode is relevant for practical batch processing because CUDA runtime initialization can be amortized over multiple images.

Both implementations were compiled with `-O3` and `-march=native`; the CUDA build additionally uses `-use_fast_math`. Image loading takes place before the total timer is started, and writing the final overlay image takes place after the timer is stopped. The measured total therefore includes preprocessing, segmentation, contour and signal processing, classification, and rendering of the overlay, but excludes file input and output.

The CSV files contain host-side wall-clock timings. The CUDA results distinguish `cuda_setup_ms` from the processing stages, but do not provide a separate decomposition into host-to-device transfer, pure kernel execution, and device-to-host transfer. Consequently, the CUDA stage values must be interpreted as pipeline-stage wall times rather than isolated kernel times. The end-to-end speedup is calculated as

$$S = \frac{\widetilde{T}_{\text{serial,total}}}{\widetilde{T}_{\text{CUDA,total}}}, \tag{1}$$

where $\widetilde{T}$ denotes the median wall-clock time.

15.2 Overall runtime and scaling

Resolution	Serial [ms]	CUDA [ms]	Speedup
1024×1409	108.593	179.539	0.61×
2048×2817	449.021	212.454	2.11×
4096×5635	2159.129	263.383	8.20×
5100×7016	3499.812	290.291	12.06×

Table 3: Median end-to-end runtime in single-image mode. Each entry first aggregates ten runs per image and then takes the median over 13 images.

The smallest images do not provide enough work to compensate for CUDA initialization and allocation overhead. At 1024×1409, the CUDA implementation is approximately $1/0.61 = 1.65$ times slower than the serial implementation. The crossover occurs between the first and second test case: from 2048×2817 onward, the CUDA pipeline is faster. At the largest resolution, the measured median speedup reaches 12.06×.

The different scaling behaviour is also visible in the stage breakdown shown in Figure 2. The serial runtime grows from approximately 109 ms to 3.50 s and is dominated by preprocessing and connected-component labeling. The single-image CUDA runtime grows much more slowly, from approximately 180 ms to 290 ms. Its largest contribution is `cuda_setup`, which includes CUDA runtime initialization, device-buffer allocation, and upload of the input image.

15.3 Influence of component and contour complexity

Figure 3 uses only the 5100×7016 data set and orders the images by their manually counted number of puzzle pieces. The total runtime does not increase monotonically with the piece count. Instead, most of the variation between images occurs in the connected-component labeling stage.

The cost of connected-component labeling depends on the topology of the complete binary foreground image. Large or irregular regions, narrow connections, and numerous local label equivalences increase the number of labeling and union-find operations. Consequently, images with a similar number of puzzle pieces can produce substantially different workloads. This variation is particularly pronounced in the serial implementation. In the CUDA implementation, block-based union–find reduces the absolute runtime, although connected-component labeling remains the largest processing stage after CUDA setup.

Contour geometry affects the subsequent per-piece stages. Longer and more irregular boundaries produce additional contour samples for boundary extraction, contour tracing, smoothing, radial-signal generation, and peak detection. Their contribution to the measured runtime variation is smaller than that of connected-component labeling.

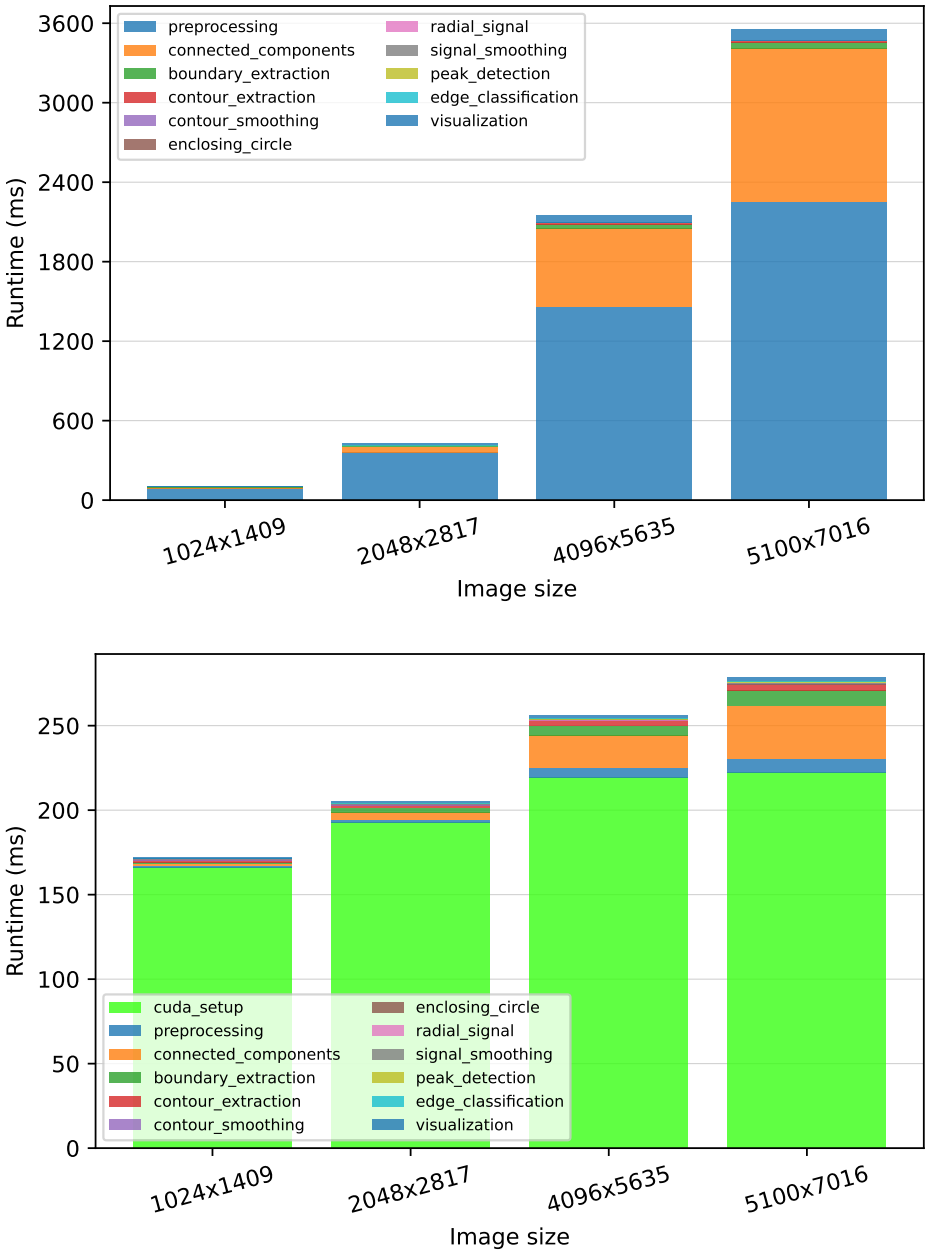


Figure 2: Median stage runtimes over increasing image resolution for the serial implementation (top) and the CUDA implementation (bottom).

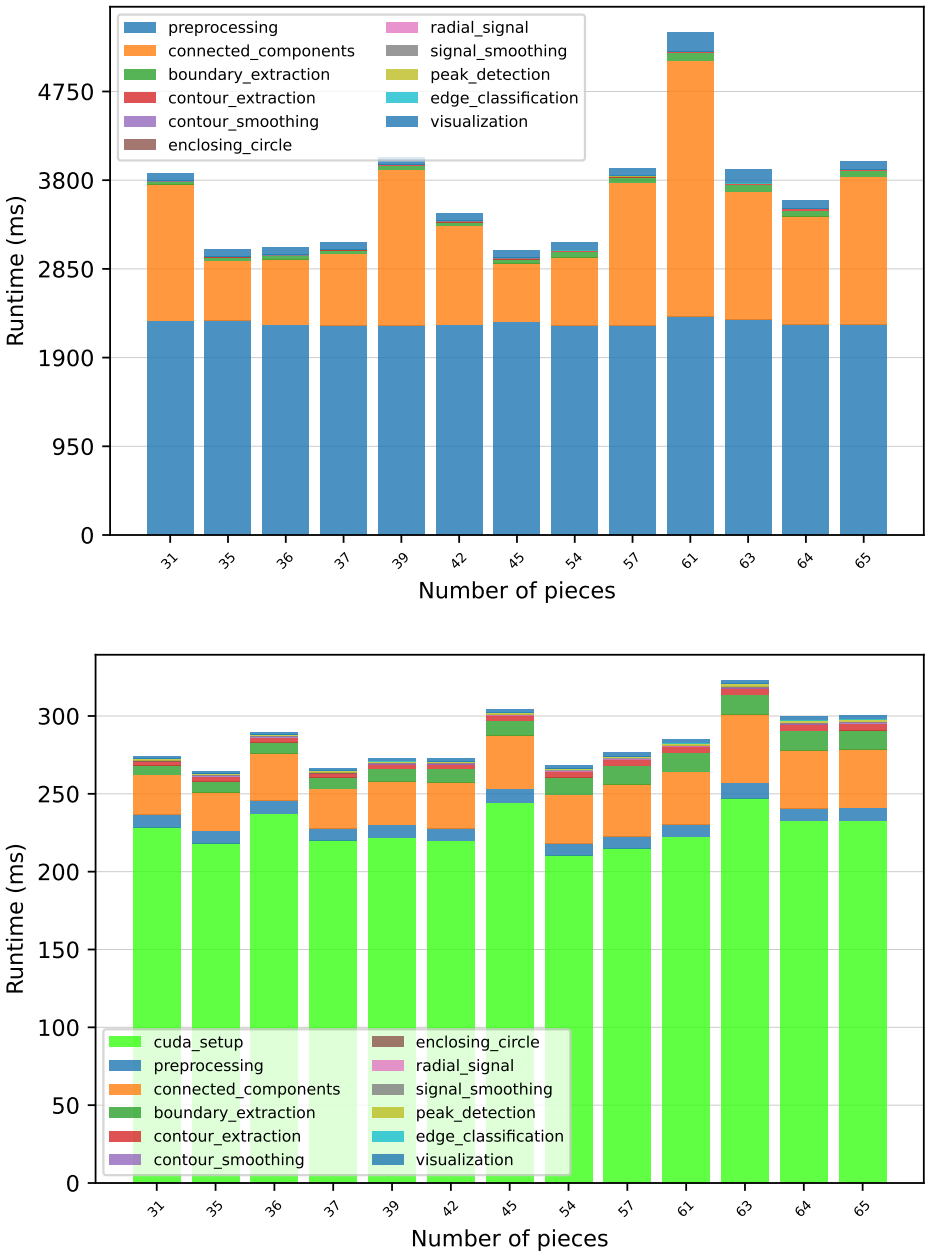


Figure 3: Stage runtimes for images ordered by their number of puzzle pieces at 5100×7016. The serial implementation is shown on top and the CUDA implementation on the bottom.

15.4 Pipeline-stage analysis

Table 4 reports the median stage durations for the largest resolution in single-image mode. CUDA setup has no serial counterpart and is therefore listed without a stage speedup.

Pipeline stage	Serial [ms]	CUDA [ms]	Speedup
CUDA setup	–	223.499	–
Preprocessing	2262.944	6.563	344.80×
Connected components	1064.926	30.083	35.40×
Boundary extraction	45.954	9.129	5.03×
Contour extraction	12.321	3.240	3.80×
Contour smoothing	0.498	0.656	0.76×
Enclosing circle	0.046	0.034	1.35×
Radial signal	0.026	0.147	0.18×
Signal smoothing	0.708	0.519	1.36×
Peak detection	0.901	0.986	0.91×
Edge classification	0.049	0.060	0.82×
Visualization	75.446	2.155	35.01×

Table 4: Median stage runtimes at 5100 × 7016 in single-image mode.

The strongest acceleration is achieved in preprocessing, where grayscale conversion, Gaussian filtering, thresholding, and morphological operations provide abundant pixel-level parallelism. Connected-component labeling also benefits strongly from the GPU-oriented block-based union-find implementation. Boundary and contour extraction show smaller but still substantial gains.

Not every stage benefits from CUDA. Contour smoothing, radial-signal generation, peak detection, and edge classification operate on comparatively small per-piece data sets or contain irregular and branch-heavy work. For these stages, launch, synchronization, and data-management overhead can exceed the amount of useful parallel work. This is consistent with the implementation: Moore-neighbour contour tracing remains CPU-side, peak filtering is only partially parallelized, and the final edge classification is batched on the CPU.

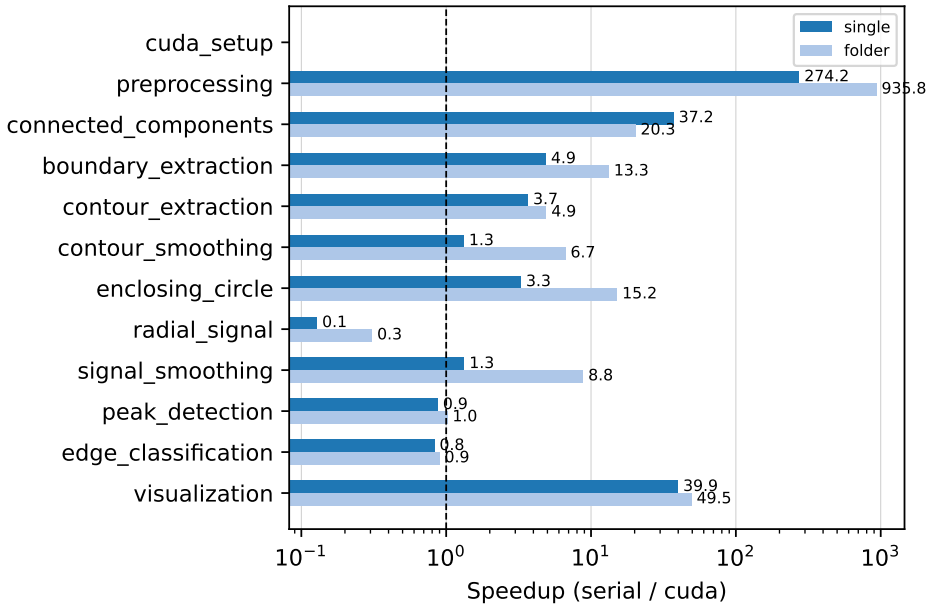


Figure 4: Per-stage speedup at 5100 × 7016. The dashed line marks break-even performance. The plot compares single-image and folder mode.

15.5 Single-image and folder mode

At the largest resolution, the median CUDA setup time is 223.499 ms in single-image mode but only 22.462 ms per image in folder mode. The median total CUDA runtime consequently decreases from 290.291 ms to 93.151 ms per image. The corresponding serial folder-mode median is 3471.432 ms, resulting in a 37.27× end-to-end speedup for batch processing. This difference shows that process and CUDA-context lifetime are central to the practical performance of the application.

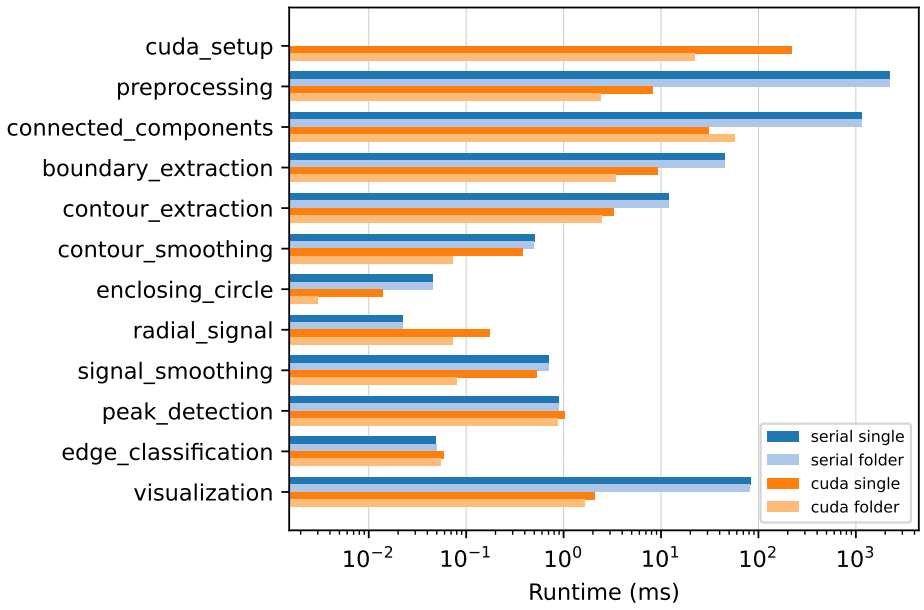


Figure 5: Stage-runtime comparison between serial and CUDA execution in single-image and folder mode at 5100×7016. The horizontal axis is logarithmic.

16 Results & Conclusion

The benchmark confirms that the CUDA implementation is advantageous once the image contains enough pixels to amortize initialization and memory-management costs. The serial implementation remains preferable for the smallest tested resolution, while the CUDA implementation reaches a median speedup of 2.11×, 8.20×, and 12.06× for the three larger resolutions. When multiple high-resolution images are processed in one program invocation, the measured speedup increases to 37.27× because CUDA initialization is shared across the batch.

Most of the overall gain originates from preprocessing and connected-component labeling, the two dominant stages of the serial pipeline. Small and irregular per-piece stages provide little or no acceleration, but their absolute runtime is low and therefore does not limit the optimized pipeline. The main remaining performance cost in single-image mode is CUDA setup. Reusing a persistent CUDA context and preallocated device buffers, as approximated by folder mode, is therefore the most promising direction for further optimization.

The comparison by puzzle-piece count also shows that the number of detected pieces is only an approximate workload indicator. The largest runtime differences originate from connected-component labeling and are better explained by foreground topology, including component area, irregular boundaries, narrow connections, and label equivalences. Contour complexity adds further variation in the later stages, but its absolute contribution is smaller. A more detailed workload characterization should therefore include foreground area, labeling complexity, and total contour length in addition to the number of pieces.

The qualitative classification examples show a similar distinction between improvements that were solved by better preprocessing and limitations that remain in the contour and edge-classification logic. Before arriving at the current result, several segmentation parameters were tested on difficult examples like those in Figure 6. The most important changes were keeping morphological opening instead of switching to closing, using a stronger Gaussian blur with a 5×5 kernel and $\sigma = 1.4$, applying a negative Otsu offset of -15, and increasing the reference minimum component area to 45000 pixels. Closing was tested because it could remove small gaps, but it also produced worse topology errors, such as merged pieces and contour shortcuts. The final tuning therefore tries to reduce small boundary gaps while keeping the topology of the detected pieces stable.

In the first example, the new result fixes an edge misclassification that appeared in the older output. The second example is more subtle: the final class is correct, but part of the printed number `5` is still interpreted as contour structure. The detail view in Figure 7 shows why this happens, because the dark marking lies very close to the exterior boundary in the thresholded image. The third example shows the opposite problem. The contour is extracted reasonably well, but the piece is still assigned to the wrong class, which means that not all remaining errors are caused by segmentation alone.

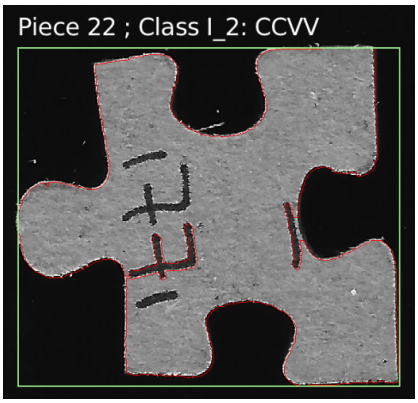
The results also demonstrate why end-to-end measurements are necessary. Purely reporting fast pixel kernels would hide the substantial setup cost for small inputs. Including setup, CPU-side stages, and rendering yields a more realistic assessment: GPU acceleration is not universally faster, but it provides large and reproducible benefits for medium and large images and especially for batched workloads.

17 Future Work

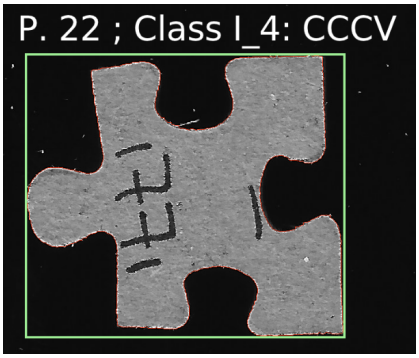
The remaining performance potential is mainly in the pipeline around the kernels, not only in the kernels themselves. In single-image mode, CUDA setup, device allocations, scratch-buffer allocation, and input copy still form a large part of the measured CUDA runtime. Folder mode shows that this cost becomes much less important when several images are processed in one program invocation. A future version should therefore use a long-lived pipeline object that initializes CUDA once and reuses device and scratch buffers across images. This would also make the speedups of the individual kernels more visible, because the one-time setup cost would no longer dominate each image.

Streaming is the natural next step for batch processing. With reusable buffers, CUDA streams and double buffering could overlap CPU image loading, host-to-device copies, GPU computation, device-to-host copies, and remaining CPU-side stages for different images. Pinned memory should only be reconsidered in this context, because one-shot pinned allocations were tested and did not give a convincing improvement. CUDA startup latency could also be hidden during CPU image loading by initializing the

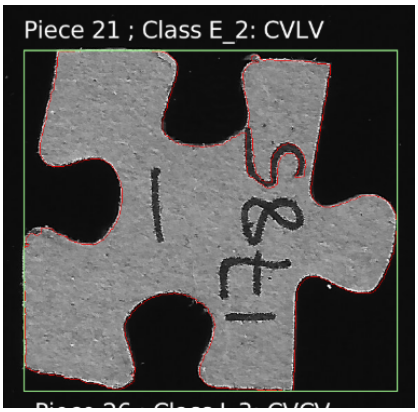
Older result

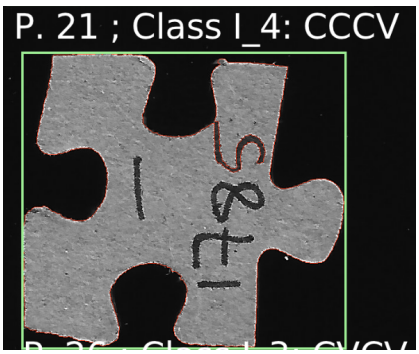


New result

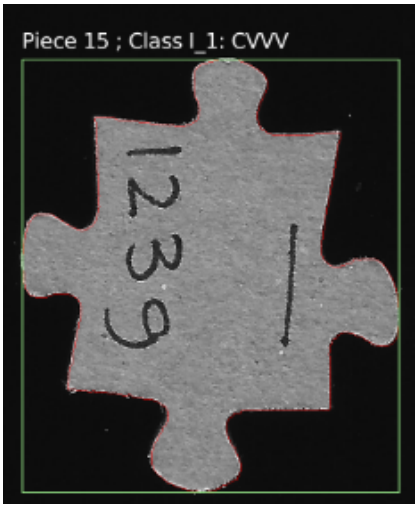


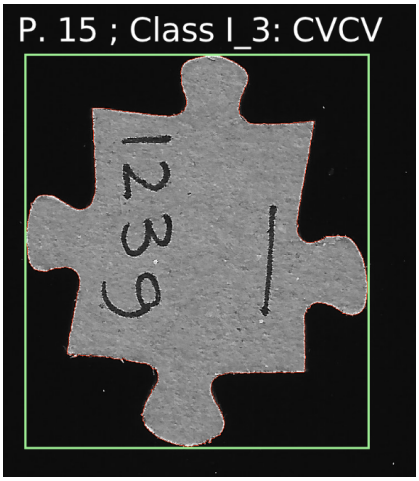
Case 1: edge misclassification improved after tuning.





Case 2: the class is correct, but the printed 5 still affects the contour.





Case 3: the contour is reasonable, but the final class is still wrong.

Figure 6: Before-and-after classification examples. Each row compares an older result with the current result for the same difficult piece.

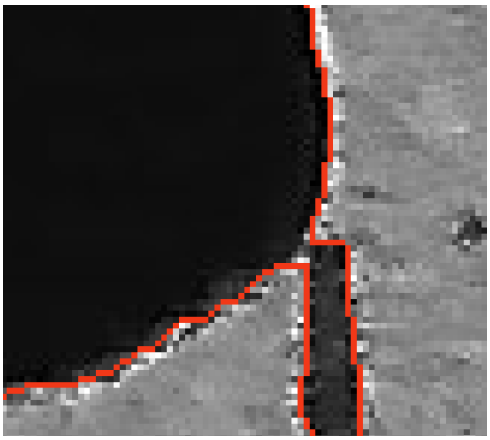


Figure 7: Detail view of the second example. The printed `5` is close enough to the piece boundary that the global thresholding pipeline can include it as part of the detected contour.

CUDA context in the background. More advanced input paths, such as nvJPEG or GPUDirect Storage, could be evaluated later, but they depend on file format and hardware support.

Another improvement would be to keep more intermediate data on the GPU. The current pipeline still copies some data back because later stages are CPU-side. The radial signal is a good example: optimizing only the distance kernel is not very useful if the result is immediately copied to the host for later analysis. An isolated GPU edge-classification experiment also showed this problem, since the saved copy was outweighed by a small launch, tiny transfers, and packing overhead. A better future design would move peak filtering, corner selection, and edge classification together, so that the whole signal-analysis path can stay on the device and only compact final results are copied back. Moore contour tracing is more difficult, because the current algorithm is sequential; a GPU version would likely require a different contour-ordering algorithm.

Inside the CUDA stages, further gains could come from kernel fusion and better memory reuse. The preprocessing path already fuses binarization with erosion and uses shared-memory tiles for morphology. A shared-memory tiled Gaussian blur or more fused preprocessing stages could reduce repeated global-memory reads and intermediate full-image writes. Connected-component labeling and region construction are harder, because union-find root chasing and component-based atomics are inherently irregular, but shared-memory or warp-level reductions could still reduce atomic contention in the region-statistics step.

Memory layout could also be investigated further. Most image-wide kernels already use flat row-major arrays, but the RGB input is still interleaved three-byte data. An aligned RGBA or `uchar4`-style layout could improve input loads, and block shapes such as `32x8` could be tested for more row-wise coalescence. For contour and signal data, a struct-of-arrays layout could be tested internally on the GPU side, while keeping the public `CoordinateVector` interface unchanged for the serial code and tests.

Finally, there is still room to improve robustness. The current segmentation is based on global thresholding, Gaussian smoothing, and morphological opening. This works well after tuning, but dark printed labels or marks close to a piece boundary can still distort the contour. Future versions could test adaptive thresholding, hole filling, mark suppression inside detected components, or a small contour-repair step. The before-and-after examples in the conclusion show some of these limitations visually, but a larger error analysis would still help to separate segmentation errors from later edge-classification errors.

References

[1] Stefano Allegretti, Federico Bolelli, Michele Cancilla, and Costantino Grana. A block-based union-find algorithm to label connected components on gpus. In Elisa Ricci, Samuel Rota Bulò, Cees Snoek, Oswald Lanz, Stefano Messelodi, and Nicu Sebe, editors, *Image Analysis and Processing – ICIAP 2019*, pages 271–281, Cham, 2019. Springer International Publishing.